PROJECT DOCUMENTATION

Smart Alumni Networking and Career Support System

A Data Structures and Algorithms Project Report

Submitted by

Ajay Kumar Gupta

Program: B.Tech Computer Science and Engineering

Course: Data Structures and Algorithms – II (CCSE0301)

Faculty Guide: Mr. Shamshad Ali

Table of Contents

1. Abstract
2. Introduction
3. Problem Statement
4. Objectives
5. Target Users and Stakeholders
6. Literature Review
7. Requirements Analysis
8. System Design and Architecture
9. Data Structures Used
10. Complexity Analysis
11. Implementation Plan and Module Breakdown
12. Expected Outcomes
13. Advantages and Limitations
14. Future Scope
15. Conclusion
16. References

1. Abstract

The Smart Alumni Networking and Career Support System is a conceptual software solution designed to bridge the communication gap between current students and alumni of an institution. As an alumni network grows over the years, the volume of records — covering names, graduation years, branches, skills, employers, job roles, locations and areas of expertise — becomes too large to search and manage manually. This project studies how core data structures and algorithms, specifically trees, self-balancing trees, heaps and graphs, can be applied to design an efficient backend for storing alumni information, searching records, prioritizing the most relevant mentors, and modelling the web of relationships that connects students, alumni, skills and organisations.

The system is organised around four pillars: a Binary Search Tree / AVL Tree layer for fast, ordered storage and retrieval of alumni records; a heap-based priority layer for ranking alumni by relevance to a student's request; a graph layer (built using adjacency lists and adjacency matrices) for representing and traversing the student–alumni–company network; and a set of supporting algorithms such as BFS, DFS and heap sort that operate on these structures. The document explains the motivation, design, algorithmic approach, complexity analysis and expected outcomes of the system, and lays out a roadmap for turning the design into a working implementation.

2. Introduction

Every academic institution builds, year after year, a growing community of graduates who move into diverse industries, roles and locations. This community is a valuable resource: alumni can mentor current students, refer them for internships and jobs, share domain expertise, and offer guidance on career paths. In practice, however, most institutions manage alumni information through spreadsheets, social media groups or informal contact lists. These approaches do not scale — as the alumni base grows into the thousands, finding "an alumnus who works in cloud computing at a mid-size company in Bangalore and is willing to mentor final-year students" becomes a slow, manual and unreliable process.

The Smart Alumni Networking and Career Support System addresses this gap by treating alumni management as a data structures problem. Instead of a flat list, alumni records are organised in tree structures keyed on attributes such as Alumni ID, enabling logarithmic-time search when the tree is balanced. Requests for mentorship are treated as a ranking problem solved with heaps, so that the most relevant alumni surface first. Relationships between people, skills and companies are modelled as a graph, allowing the system to answer questions such as "which alumni are reachable from this student through a chain of shared skills or companies" using standard graph traversal algorithms.

2.1 Motivation

The motivation for this project comes from a simple observation: institutions already have rich data about their alumni, but very little structure around it. A well-designed data layer, built on classical data structures, can turn a static list of names into a searchable, rankable and traversable network — without requiring any change in the underlying information, only in how it is organised and accessed.

2.2 Scope of the Project

The scope of this documentation covers the design of the data model, the choice and justification of data structures for each sub-problem (searching, ranking, and relationship-mapping), the algorithms used to operate on those structures, and an analysis of their time and space complexity. Full-scale development of a production web or mobile application, user authentication, and deployment infrastructure are treated as extensions beyond the current scope, and are discussed briefly under Future Scope.

3. Problem Statement

Design and conceptually implement a Smart Alumni Networking and Career Support System that helps students discover alumni suited to their mentorship, internship or career-guidance needs, by organising alumni information and relationships using tree, heap and graph-based data structures.

3.1 Problem Context and Background

Alumni records typically include attributes such as graduation year, branch, technical skills, current company, job role, geographic location and domain expertise. As the number of alumni increases, linear or manual searching across these attributes becomes inefficient. A structured system built on appropriate data structures can provide faster searching, meaningful prioritisation of results, and systematic management of the relationships between students and alumni.

Binary search trees and their self-balancing variants (AVL trees) are well suited to organise and search alumni records by a unique key. Heaps are suited to prioritisation problems, such as ranking alumni by how closely they match a student's mentorship request. Graphs are a natural fit for representing the network of relationships between students, alumni, skills and employers, since these relationships are inherently many-to-many rather than hierarchical.

3.2 Problem Identification

- Students often cannot easily locate alumni who match a specific skill, company, job role or mentoring requirement.
- The system needs to store and search alumni records efficiently even as the alumni network grows into the thousands.
- Relationships among students, alumni, skills and companies form a dense network that is difficult to represent using simple lists or tables.
- Requests for mentorship need to be ranked by relevance rather than returned in an arbitrary order, which calls for a priority-based mechanism.

4. Objectives

1. To design a data model for alumni and student records that supports efficient insertion, search and update operations.
2. To apply Binary Search Tree (BST) and AVL Tree concepts to keep alumni records organised and searchable by a unique key such as Alumni ID.
3. To use min-heap and max-heap structures, along with heap sort, to rank alumni by a computed relevance or priority score for a given mentorship request.

4. To model the student–alumni–company–skill network as a graph, and to compare adjacency list and adjacency matrix representations for this use case.
5. To apply Breadth-First Search (BFS) and Depth-First Search (DFS) for discovering connections and paths within the alumni network.
6. To analyse the time and space complexity of each chosen data structure and algorithm in the context of this application.
7. To outline a practical software architecture through which these data structures can be exposed as a usable career-support system.

5. Target Users and Stakeholders

The system is designed around four broad groups of stakeholders, each of whom interacts with the alumni network in a different capacity.

Stakeholder	Role in the System
Students	Search for suitable alumni for mentorship, internships, referrals and career guidance.
Alumni	Maintain personal and professional profiles, and connect with students or other alumni.
Faculty / Alumni Coordinators	Manage alumni data, verify profiles and oversee networking activities.
Placement and Career Support Teams	Use alumni information for placement drives, referrals and career-related support.
Institution / Administration	Track alumni engagement and use aggregated data for institutional planning.

6. Literature Review

Before finalising the design approach, related work in the area of alumni data management and social/professional network platforms was reviewed. The review focused on how existing systems structure alumni data and represent relationships, in order to identify data-structure choices that are appropriate for this project.

6.1 Digital Platforms for Alumni Data Management and Career Analytics

Studies on centralised alumni data platforms highlight that institutions benefit significantly from consolidating alumni records into a single searchable system, rather than maintaining scattered spreadsheets or social media groups. Centralised platforms are shown to improve the accuracy of career analytics and make it easier to track alumni outcomes over time, which supports the case for a structured, searchable backend rather than an unstructured record set.

6.2 Alumni Information Management Systems in Universities

Research on alumni information systems deployed at universities emphasises structured record-keeping — organised by attributes such as graduation batch, department and employment status — as a way to make retrieval faster and more reliable as the alumni base grows. This supports the choice of a key-based, tree-organised storage model in the present project, since it allows records to be retrieved in sorted or ranged order efficiently.

6.3 Intelligent Alumni Network Platforms and Interaction Modelling

Work on intelligent alumni network platforms treats alumni interactions explicitly as a network or graph problem, where nodes represent individuals and edges represent relationships such as shared institution, shared employer or mentorship links. This reinforces the decision, in the present project, to model student–alumni relationships using graph structures rather than flat relational tables, since graphs naturally support neighbourhood queries and path discovery that a table-based design would need extra logic to support.

6.4 Summary of Findings

Across the reviewed literature, two consistent themes emerge: first, centralised and structured storage of alumni data considerably improves search and analytics compared to unstructured record-keeping; second, the relationships between alumni, students and organisations are best represented as a network rather than a hierarchy. These findings directly informed the two-layer design used in this project — a tree/heap layer for record storage and ranking, and a graph layer for relationship modelling.

7. Requirements Analysis

7.1 Functional Requirements

- The system shall allow addition, update and removal of alumni and student profile records.
- The system shall support searching alumni records by unique identifier, skill, company, job role or location.
- The system shall rank matching alumni by a computed relevance score for a given student request.

- The system shall represent and allow traversal of the network of relationships between students, alumni, skills and companies.
- The system shall support discovery of indirect connections, such as a path from a student to an alumnus through a common skill or shared contact.

7.2 Non-Functional Requirements

- Search operations should remain efficient (logarithmic or near-logarithmic time) as the number of alumni records grows.
- The system should remain responsive even when the underlying network has a large number of relationships (edges).
- The design should be modular, so that the storage layer, ranking layer and network layer can be developed, tested and scaled independently.
- The data structures used should be memory-efficient, particularly the representation chosen for the relationship graph.

8. System Design and Architecture

The system is designed in three cooperating layers, each backed by a specific family of data structures. This separation keeps the responsibilities of storage, ranking and relationship modelling independent, which makes the design easier to reason about, test and extend.

8.1 Layered Architecture

- Storage Layer — a Binary Search Tree / AVL Tree keyed on Alumni ID (or another unique attribute) for organised storage and efficient search, insertion and deletion of alumni records.
- Ranking Layer — a heap-based priority structure that computes and maintains a relevance score for alumni against a given student request, and returns the top matches.
- Network Layer — a graph, represented using an adjacency list or adjacency matrix, that models relationships between students, alumni, skills and companies, and supports traversal algorithms such as BFS and DFS.

8.1.1 High-Level Data Flow

A student request (for example, "find alumni working in data analytics in Pune willing to mentor") first queries the storage layer to filter candidate alumni matching the hard constraints (skill, location). The filtered candidates are then passed to the ranking layer, where a max-heap orders them by a computed relevance score. Finally, the network layer can be queried to check how closely connected a given alumnus is to the student — for instance, through a common contact, shared skill node, or shared past project — before a final ranked and annotated result list is returned.

8.2 Data Model

Each Alumni record is modelled with the following attributes, which serve as the fields on which the tree, heap and graph structures operate:

Attribute	Description
Alumni ID	Unique key used to organise records in the BST / AVL tree.
Name	Full name of the alumnus.
Graduation Year	Year of graduation, usable as a secondary search / filter key.
Branch	Academic branch or department.
Skills	Set of technical or domain skills, used for matching and as graph edges.
Company & Role	Current employer and job title, used for filtering and matching.
Location	Current city / region, used for location-based filtering.
Priority Score	Computed relevance score used as the heap key when ranking matches.

9. Data Structures Used

This section describes each data structure used in the design, why it was chosen for its specific role, and its behaviour in this application.

9.1 Binary Search Tree (BST)

A Binary Search Tree stores alumni records so that, for any node, all records in the left subtree have a smaller key (e.g., Alumni ID) and all records in the right subtree have a larger key. This ordering allows the system to search, insert or delete a record by repeatedly halving the search space, which is far faster than scanning a flat list once the alumni base grows large.

In the context of this project, the BST forms the base storage structure for alumni records. Search operations such as "find the alumnus with this ID" or "list all alumni with ID in this range" (useful for batch-wise queries) map naturally onto standard BST search and in-order traversal operations.

9.1.1 Key BST Operations (Pseudocode)

```
function SEARCH(node, key):
    if node is NULL or node.key == key:
        return node
    if key < node.key:
        return SEARCH(node.left, key)
    else:
        return SEARCH(node.right, key)

function INSERT(node, record):
    if node is NULL:
        return new Node(record)
    if record.key < node.key:
        node.left = INSERT(node.left, record)
    else:
        node.right = INSERT(node.right, record)
    return node
```

9.2 AVL Tree

A plain BST can become skewed (in the worst case, behaving like a linked list) if alumni records are inserted in sorted order, which degrades search time to O(n). An AVL tree solves this by keeping the tree height-balanced: after every insertion or deletion, the balance factor of each node (the height difference between its left and right subtrees) is checked, and rotations (single or double, left or right) are applied whenever this factor exceeds 1.

Using an AVL tree instead of a plain BST guarantees that alumni search, insertion and deletion remain O(log n) even in the worst case, which matters for a system that is expected to grow continuously as new batches of students graduate.

9.2.1 Balancing via Rotations

- Left-Left (LL) Case — corrected with a single right rotation.
- Right-Right (RR) Case — corrected with a single left rotation.
- Left-Right (LR) Case — corrected with a left rotation followed by a right rotation.
- Right-Left (RL) Case — corrected with a right rotation followed by a left rotation.

9.3 Heaps (Min-Heap / Max-Heap) and Heap Sort

A heap is a complete binary tree, usually stored as an array, that maintains the heap property: in a max-heap, every parent node's key is greater than or equal to its children's keys (the opposite for a min-heap). This project uses a max-heap to keep the alumnus with the highest relevance score at the root, so that

the best match for a student's request can always be retrieved in O(1) time, with re-heapification after each removal taking O(log n).

Heap Sort, built on top of the heap structure, is used when the system needs a fully sorted list of candidate alumni by priority score — for example, to display a ranked results page rather than only the single best match.

9.3.1 Heap Operations (Pseudocode)

```
function INSERT_HEAP(heap, record):
    heap.append(record)
    i = heap.size - 1
    while i > 0 and heap[parent(i)].score < heap[i].score:
        swap(heap[parent(i)], heap[i])
        i = parent(i)

function EXTRACT_MAX(heap):
    top = heap[0]
    heap[0] = heap[last]
    remove last element
    HEAPIFY_DOWN(heap, 0)
    return top
```

The priority score used as the heap key can be computed from a weighted combination of factors such as skill match, company/domain match, location match and alumnus availability for mentoring, giving the system flexibility to tune what "best match" means.

9.4 Graphs (Adjacency List and Adjacency Matrix)

The relationships between students, alumni, skills and companies are modelled as a graph, where each entity is a node and each relationship (e.g., "mentors", "shares skill with", "works at") is an edge. Two standard representations were compared for this network:

Representation	Space Complexity	Edge Check	Best Suited For
Adjacency Matrix	$O(V^2)$	O(1)	Dense networks with frequent edge-existence checks.
Adjacency List	O(V + E)	O(degree of V)	Sparse networks, such as a typical alumni graph, where most nodes connect to only a few others.

Because a typical alumni network is sparse — most alumni are connected to a relatively small number of students, skills or companies compared to the total size of the network — the adjacency list representation is preferred as the primary structure, with the adjacency matrix considered only for smaller, densely connected sub-graphs (for example, within a single graduating batch).

9.4.1 Graph Traversal: BFS and DFS

Breadth-First Search (BFS) is used to find the shortest chain of connections between a student and a potential mentor (for example, "alumnus of alumnus"), since BFS explores the network level by level. Depth-First Search (DFS) is used for tasks such as discovering all alumni reachable from a given skill or company node, where exploring one branch fully before backtracking is more natural.

```
function BFS(graph, start):
    visited = {start}
    queue = [start]
    while queue is not empty:
        node = queue.dequeue()
        process(node)
        for neighbor in graph.adjacency[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.enqueue(neighbor)
```

10. Complexity Analysis

The following table summarises the time complexity of the core operations used across the system, comparing best-case and worst-case behaviour.

Data Structure	Operation	Best Case	Worst Case
Binary Search Tree	Search / Insert / Delete	O(log n)	O(n) — when unbalanced
AVL Tree	Search / Insert / Delete	O(log n)	O(log n) — always balanced
Max-Heap / Min-Heap	Insert	O(1)	O(log n)
Max-Heap / Min-Heap	Extract Max/Min	O(log n)	O(log n)
Heap Sort	Sort n records	O(n log n)	O(n log n)
Graph (Adjacency List)	BFS / DFS Traversal	O(V + E)	O(V + E)
Graph (Adjacency Matrix)	Edge Existence Check	O(1)	O(1)
Graph (Adjacency List)	Edge Existence Check	O(1)	O(degree of V)

The AVL tree is chosen over a plain BST specifically because the worst-case behaviour of a plain BST — O(n) when records are inserted in a already-sorted or adversarial order — is unacceptable for a system that must remain responsive as thousands of alumni records accumulate over successive graduating batches. Similarly, the adjacency list is chosen over the adjacency matrix for the main network representation because its O(V + E) space usage is far more efficient than $O(V^2)$ for a sparse, real-world alumni network.

11. Implementation Plan and Module Breakdown

The proposed implementation is divided into independent modules, each of which can be developed and tested separately before integration.

Module	Responsibility	Core Data Structure(s)
Profile Management	Create, update and remove student and alumni profiles.	AVL Tree
Search Engine	Search alumni by ID, skill, company, role or location.	AVL Tree, Hashing (auxiliary)
Ranking Engine	Compute relevance scores and return top-matching alumni.	Max-Heap, Heap Sort
Network Engine	Model and traverse relationships between entities.	Graph (Adjacency List)
Reporting / Analytics	Summarise engagement statistics for administrators.	In-order traversal, Graph metrics

11.1 Development Roadmap

8. Phase 1 — Finalise data fields, build the AVL-tree based storage layer, and implement core search operations.

9. Phase 2 — Implement the priority-score formula and the heap-based ranking engine; validate using heap sort against sample data.
10. Phase 3 — Build the graph model using adjacency lists, and implement BFS/DFS-based relationship queries.
11. Phase 4 — Integrate the three layers behind a unified query interface and test with realistic, larger sample datasets.
12. Phase 5 — Evaluate performance, refine the ranking formula, and document results.

12. Expected Outcomes

- A working demonstration of alumni record storage and search using an AVL tree, with measured search times compared against a plain BST on skewed input.
- A working demonstration of the ranking engine, showing how a max-heap surfaces the most relevant alumni for a sample student request.
- A graph-based visualisation or trace of BFS/DFS traversal showing how a student can be connected to a suitable alumnus through the network.
- A comparative analysis of adjacency list versus adjacency matrix performance on a representative alumni network size.
- A consolidated understanding of how classical data structures from a DSA curriculum can be mapped onto a realistic, socially useful application.

13. Advantages and Limitations

13.1 Advantages

- Search and update operations remain efficient even as the alumni database grows, due to the use of self-balancing trees.
- Ranking by relevance, rather than arbitrary or alphabetical order, improves the usefulness of search results for students.
- Representing relationships as a graph allows the system to answer questions about indirect connections that a flat database would struggle with.
- The modular, layered design allows each data structure to be replaced or optimised independently as requirements evolve.

13.2 Limitations

- The current design is conceptual and algorithm-focused; it does not yet include a persistent database, user interface or authentication layer.
- The relevance-scoring formula used for ranking is a simplified model and would need refinement using real usage data and possibly weighted or learned scoring in a production system.
- Very large, highly interconnected networks may still require distributed or database-backed graph storage beyond a single in-memory adjacency list.

14. Future Scope

- Persisting the tree, heap and graph structures to a database (relational or graph database) so that data survives beyond a single runtime session.
- Building a web or mobile front-end that exposes search, ranking and networking features to students, alumni and administrators.
- Incorporating recommendation techniques (e.g., collaborative filtering) alongside the heap-based ranking to further personalise mentor suggestions.

- Adding authentication, role-based access control and privacy controls so that alumni can manage the visibility of their own data.
- Extending the graph layer with community-detection or shortest-path algorithms (such as Dijkstra's algorithm) to identify well-connected "hub" alumni who could be effective as mentors or points of contact.

15. Conclusion

The Smart Alumni Networking and Career Support System illustrates how classical data structures — binary search trees, AVL trees, heaps and graphs — can be combined to solve a real, socially meaningful problem: connecting students with the alumni best placed to guide them. By separating the system into a storage layer (AVL tree), a ranking layer (heap) and a relationship layer (graph with adjacency-list representation), the design keeps each concern efficient and independently improvable. The complexity analysis presented in this document shows that this combination scales far better than a naive, list-based approach as the alumni network grows, while the literature review confirms that centralised, structured alumni data and network-based relationship modelling are consistent with current practice in similar systems. The next stage of this project is to translate this design into a working implementation and to validate its performance on realistic data.

16. References

B. Anthony Jnr., et al., "Digital Platform for Centralized Alumni Data Management and Career Analytics," International Research Journal of Engineering and Technology (IRJET), vol. 11, no. 8, pp. 450–458, August 2026.

MDPI Proceedings, "Development of Alumni Information Management System in a State University," MDPI Engineering Proceedings, vol. 143, no. 1, p. 38, 2026.

Z. Wang, H. Wang, and Z. Liu, "Construction of an Intelligent Alumni Network Platform and Research on Alumni Interaction," International Journal of High Speed Electronics and Systems, vol. 35, no. 2, pp. 104–119, 2026.

T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 3rd ed., MIT Press, 2009.

Course material and lecture notes, Data Structures and Algorithms – II (CCSE0301), Units 1 (Trees) and 2 (Graphs).